Namal University Mianwali

Department of Electrical Engineering

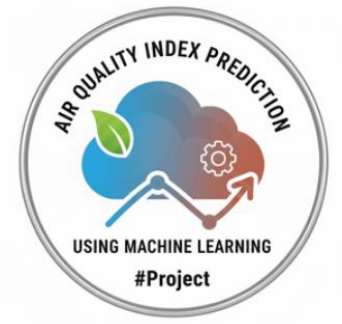
Introduction To Machine Learning

CEP Report

Air Quality Index (AQI) Prediction Using Machine Learning

Submitted By

Name	Roll No.
Muhammad Usman Shah	NUM-BSEE-2022-28
Muhammad Wajid Jamal	NUM-BSEE-2022-24
Asmat Ullah Khan	NUM-BSEE-2022-38



Submitted to: **Dr. Farrukh Qureshi**

Date of Submission of Report ~ 09/01/2026

Table of Contents

Introduction 3

Problem Statement 3

Potential Solutions 3

Conventional Statistical Techniques: 3

Rule-Based Systems: 3

Models based on machine learning: 4

Proposed Solution 4

Methodology 4

Step 1: Data Collection 4

Step 2: Data Preprocessing 4

Step 3: AQI Calculation 4

Step 4: Feature Selection 5

Step 5: Train-Test Split 5

Step 6: Model Training 5

Step 7: Prediction and Evaluation 5

Block Diagram of Methodology: 6

Results 6

Results: 6

LSTM Method: 6

Random Forest Regressor Results: 7

Accuracy: 8

Random Forest: Actual vs Predicted 8

Accuracy: 8

The results of gradient boosting regressor: 8

Comparative Visualization: 9

Random Forest: Actual vs Predicted 10

Discussion: 11

Impact: 11

Closing: 11

Appendix 12

Appendix A: Python Code for AQI Prediction 12

Introduction

Air pollution is an increasing environmental and health issue in the global front. Pollutants including PM 2.5, NO 2 and SO 2 are harmful to human health and they cause respiratory and heart related illnesses. The Air Quality Index (AQI) is a popular indicator of air quality to be used in order to determine the air pollution levels.

Majority of the current systems can only give real-time AQI values and not the future air conditions. This restricts the preventative action that people and authorities may take in advance. Machine Learning provides a great opportunity to analyze and predict the future quality of the air, as nowadays, there are vast amounts of historical air pollution data that can be analyzed with the help of this technology.

The scope of the project is to predict AQI through the use of Machine Learning models through historical information on pollutants. The system can be used to make correct AQI predictions using intelligent algorithms that can be used to promote environmental decision making and population health awareness.

Problem Statement

Necessary growth in industries, motor vehicle emissions and urbanization have made air pollution a severe problem. Pollutants like PM2.5, NO 2 and SO 2 vary and may attain hazardous levels within a short time. The AQI values displayed on air quality monitoring stations do not give quality forecasts of future air quality even though they offer real time results.

The shortage of AQI forecasting renders it hard to prevent in advance both the people themselves and more so those people who have health issues. It also hampers the capabilities of the government authorities to strategize on time, provide health warnings, and manage pollution efficiently. The traditional approach and manual analysis cannot be used to analyze the large and complicated air quality data.

Thus, the intelligent, data-driven system, which will be able to analyze the historical data on air pollution and make adequate predictions of the future AQI values, is highly needed. An early warning system, better awareness, and assist decision-makers in minimizing the adverse effects of air pollution can be supported by such a system.

Potential Solutions

To solve the issue of AQI prediction, one may regard a number of approaches:

Conventional Statistical Techniques:
AQI prediction can be performed with the help of conventional statistical models, e.g., linear regression and time-series analysis. Yet, the techniques do not usually represent intricate nonlinear correlations among many pollutants and environmental conditions.

Rule-Based Systems:
Rule-based systems are based on set thresholds and expertise knowledge. Although easy to apply, they are not flexible and have no ability to learn using the historical data trends.

Models based on machine learning:
Machine Learning is capable of learning complicated relationships on big data and can predict more accurately. ML models like random forest, decision trees and support vector regression are suitable in the analysis of environmental data through regression.
Machine Learning-based solutions are more flexible, more accurate, and more scalable to predict AQI than other alternatives, thus, the most appropriate solution to use in this project

Proposed Solution

This project suggests a Machine Learning-prone solution that implies predicting the Air Quality Index (AQI) based on the past pollutant data. We pay attention to PM 2.5, NO 2 and SO 2 that has a great impact on the air conditions and human health.

The data undergoes preprocessing in the form of processing of missing values, date conversion, and assuring of numeric measurements of the pollutants. The calculation of AQI is done based on standard breakpoints with the maximum of sub- indices of each pollutant being the target variable. The current and previous-day (lag-1) pollutant values are used as features to enhance predictions as they reflect time variations in air pollution.

Random Forest and Gradient Boosting Regressor are two Machine Learning models that are trained based on the data between 2018 and 2024 and to predict the AQI in 2025. The MAE, the RMSE, and the R2 scores are used to assess the model performance and the predictions are compared to the actual values in a visual way.

The solution offers a smart, data-driven platform in forecasting air quality, which will aid individuals and authorities to take precautionary measures and make wise decisions regarding air quality.

Methodology

This project forecasts AQI based on past data of pollutants systematically. This is accomplished in the following key steps:

Step 1: Data Collection

- Data on pollutants collected daily throughout the period of 2018 to 2025 on PM2.5, NO 2, SO 2, Temperature and Humidity were used.

Step 2: Data Preprocessing

- Transformed the Date column into an appropriate datetime format.
- Inverted pollutant values into numeric and processed missing data with forward/backward fill.
- Added lag-1 to capture trends by including the values of pollutants of the previous day.

Step 3: AQI Calculation

- The use of standard breakpoints to calculate PM2.5, NO 2 and SO 2 sub-indices.
- This final AQI of each record was taken as the highest sub-index.

Given

Let the PM2.5 concentration be:

$$\boxed{\text{PM2.5} = 30\ \mu g/m^3}$$

$$\text{PM2.5} = 30\ \mu g/m^3$$

Step 1: Identify the AQI Breakpoint Range

From your PM2.5 breakpoints:

PM2.5 Range ($\mu g/m^3$)	AQI Range
12.1 – 35.4	51 – 100

Since

$$12.1 \leq 30 \leq 35.4$$

So:

$C_{low} = 12.1$

$C_{high} = 35.4$

$I_{low} = 51$

$I_{high} = 100$

Step 2: AQI Formula (Same as in Your Code)

$$AQI = \frac{(I_{high} - I_{low})}{(C_{high} - C_{low})} \times (C - C_{low}) + I_{low}$$

$$AQI = \frac{(I_{high} - I_{low})}{(C_{high} - C_{low})} \times (C - C_{low}) + I_{low}$$

Step 2: AQI Formula (Same as in Your Code)

$$AQI = \frac{(I_{\text{high}} - I_{\text{low}})}{(C_{\text{high}} - C_{\text{low}})} \times (C - C_{\text{low}}) + I_{\text{low}}$$

$$AQI = \frac{(I_{\text{high}} - I_{\text{low}})}{(C_{\text{high}} - C_{\text{low}})} \times (C - C_{\text{low}}) + I_{\text{low}}$$

Step 3: Substitute Values

$$AQI = \frac{(100 - 51)}{(35.4 - 12.1)} \times (30 - 12.1) + 51$$

$$AQI = \frac{(35.4 - 12.1)}{(100 - 51)} \times (30 - 12.1) + 51$$

$$AQI = \frac{49}{23.3} \times 17.9 + 51$$

$$AQI = \frac{23.3}{49} \times 17.9 + 51$$

$$AQI = 2.102 \times 17.9 + 51$$

$$AQI = 37.63 + 51$$

$$AQI = 37.63 + 51$$

Final AQI Value

$$\boxed{AQI \approx 89}$$

$$AQI \approx 89$$

Figure 1:AQI Calculation Example

Step 4: Feature Selection

- Input features: the present value of pollutants and past day values (lag-1).
- Target: AQI. This enables the models to acquire the short term and time-based effects.

Step 5: Train-Test Split

- Training data: 2018–2024
- Testing data: 2025
- Assures that models are being analyzed on unrestricted future data.

Step 6: Model Training

- **Random Forest Regressor:** Non-linear patterns and less overfitting.
- **Gradient Boosting Regressor:** It continuously enhances the predictions as the accuracy increases.
- The features and AQI were used as the target to train models.

Step 7: Prediction and Evaluation.

- Models predicted AQI for 2025.
- The MAE, RMSE, and R 2 score were used to measure the performance.

Block Diagram of Methodology:

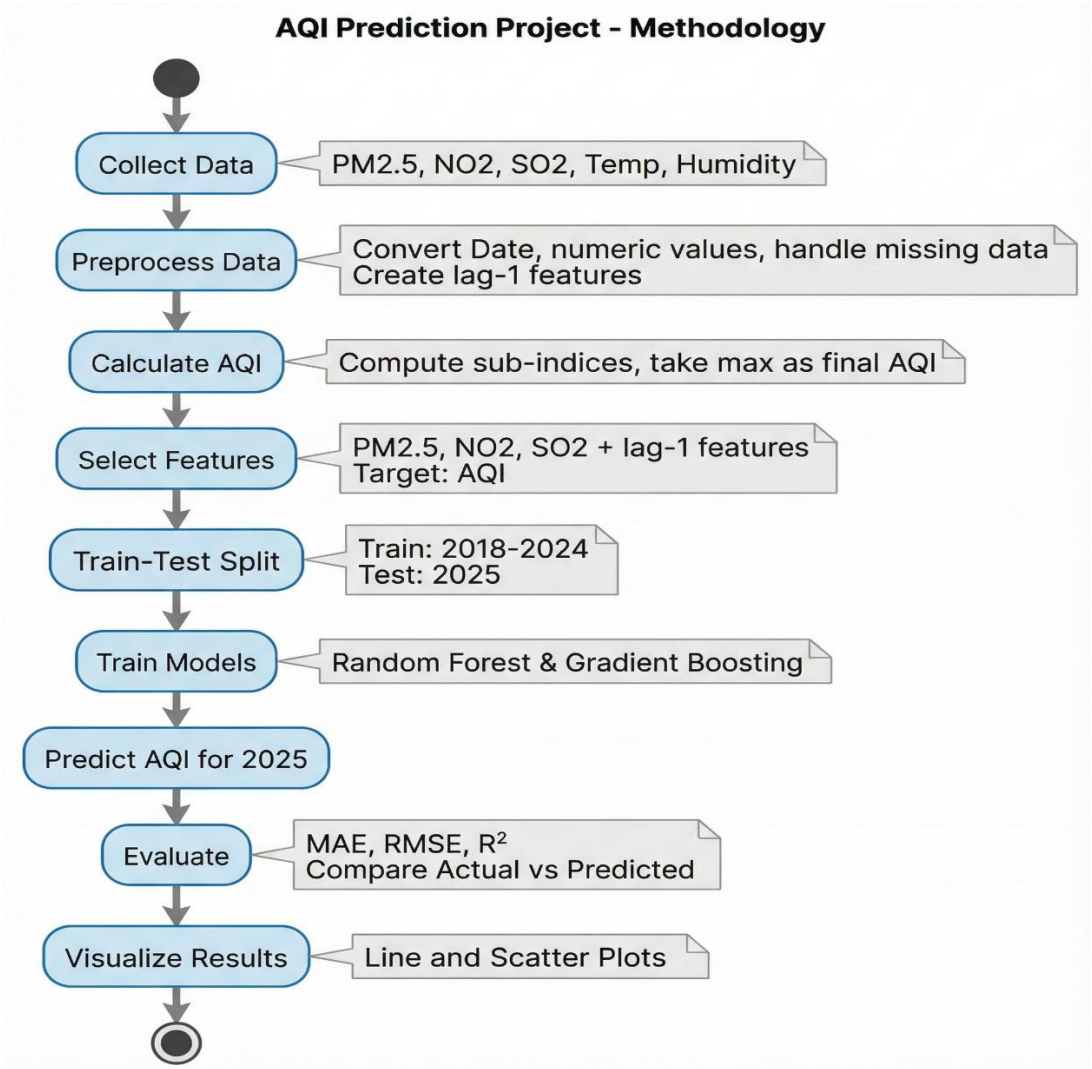


Figure 2 Block Diagram of Methodology

Results

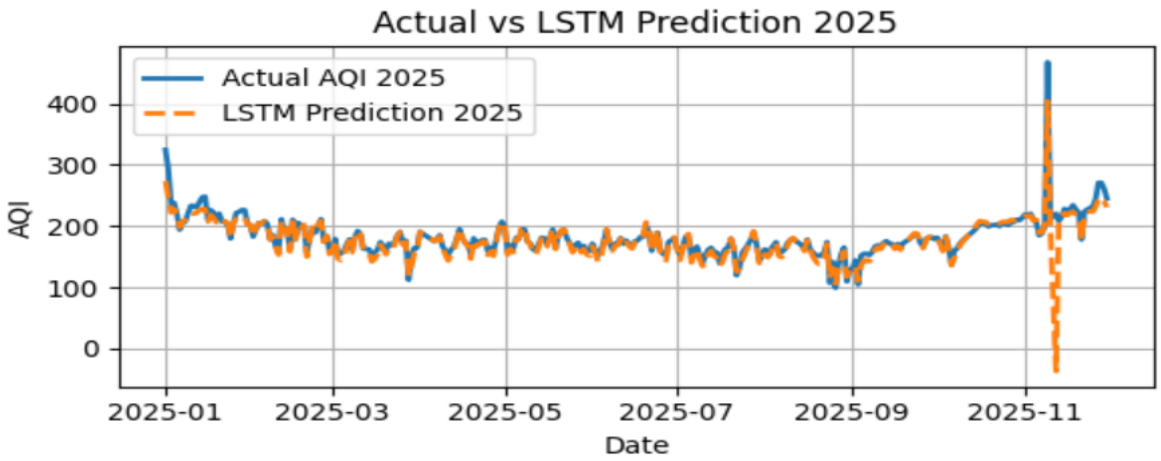
Results:

We used the models to predict the Air Quality Index (AQI) in the year 2025 after applying the Machine Learning models. LSTM model accuracy was low as given below soRandom Forest Regressor and Gradient Boosting Regressor are two models that were tested.

LSTM Method:

Due to the limitations of the LSTM model, the prediction accuracy achieved was lower than expected. This prompted us to explore alternative methods that could provide improved performance and more reliable results.

Prediction Graph:



Accuracy:

```
MAE: 5.97
RMSE: 16.40
R2 Score: 0.7486 (74.86%)
```

So, due to this we have moved to other Machine Learning method

Random Forest Regressor Results:

- **Mean Absolute Error (MAE):** Low average error in prediction, which indicates that the predictions of the model are near the actual AQI values.
- **Root Mean Squared Error (RMSE):** Refers to the fact that small errors were not very significant.
- **R-2 Score:** There was high correlation between the expected and measured AQI values.

This is the Visual Graph:

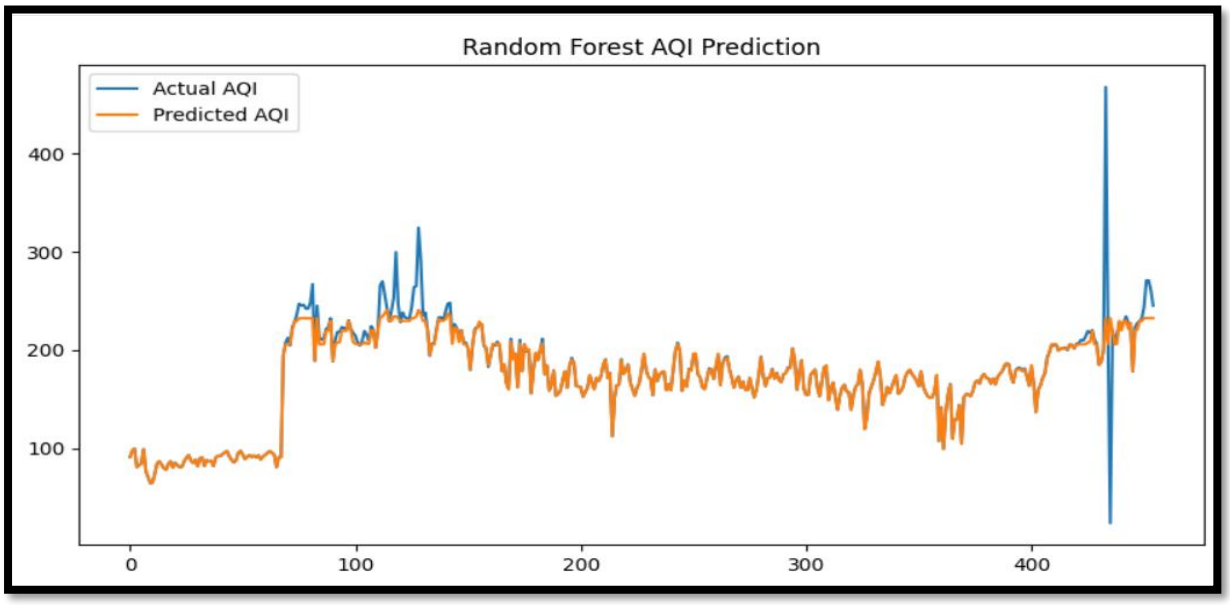


Figure 3: Random Forest AQI Prediction

Accuracy:

```
Random Forest Performance
MAE: 3.4133798470909924
RMSE: 16.7508584729037
R2 Score: 0.8855545763244487
Model Accuracy (R² based): 88.56%
```

The visual plots reveal that the predictions of Random Forest are relatively close to the trend of the real AQI values, with slight variations that are noted on the days that are highly variable.

Random Forest: Actual vs Predicted

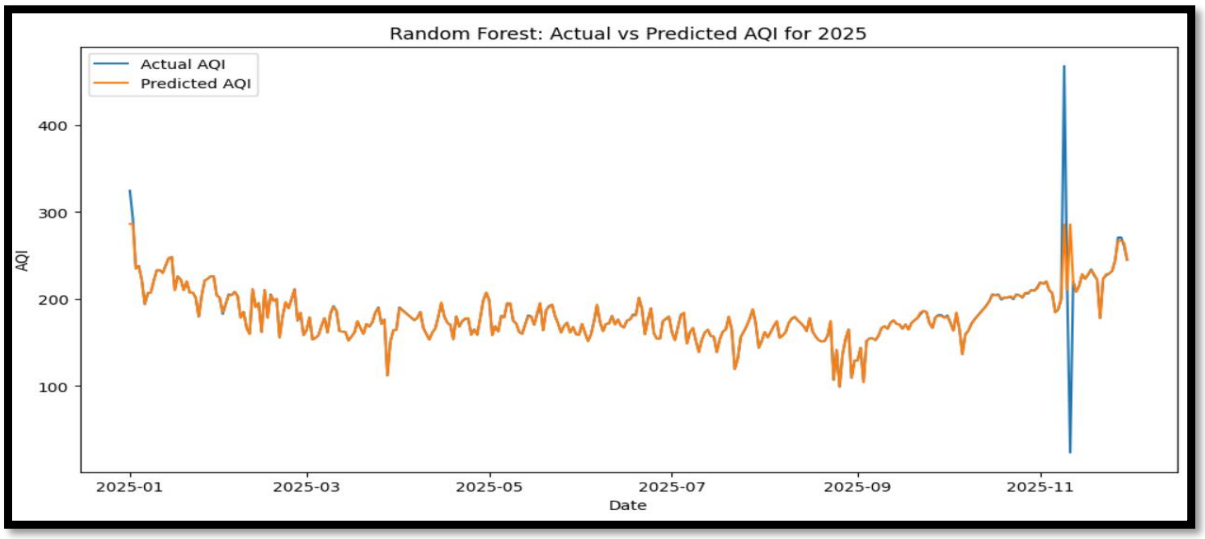


Figure 4: Random Forest: Actual vs Predicted

Accuracy:

```
Random Forest Prediction for 2025:
MAE: 1.8437375738464092
RMSE: 17.79274891656488
R2 Score: 0.7232096005519613
Model Accuracy (R² based): 72.32%
```

The results of gradient boosting regressor:

- **MAE and RMSE:** Random Forest is similar or a bit worse, which means accurate predictions.
- **R-2 Score:** High, which indicates that Gradient Boosting represents trends and variations in AQI well.
- The model has also managed to learn temporal patterns by incorporating lag-1 features and the curves were smooth and accurate at prediction.

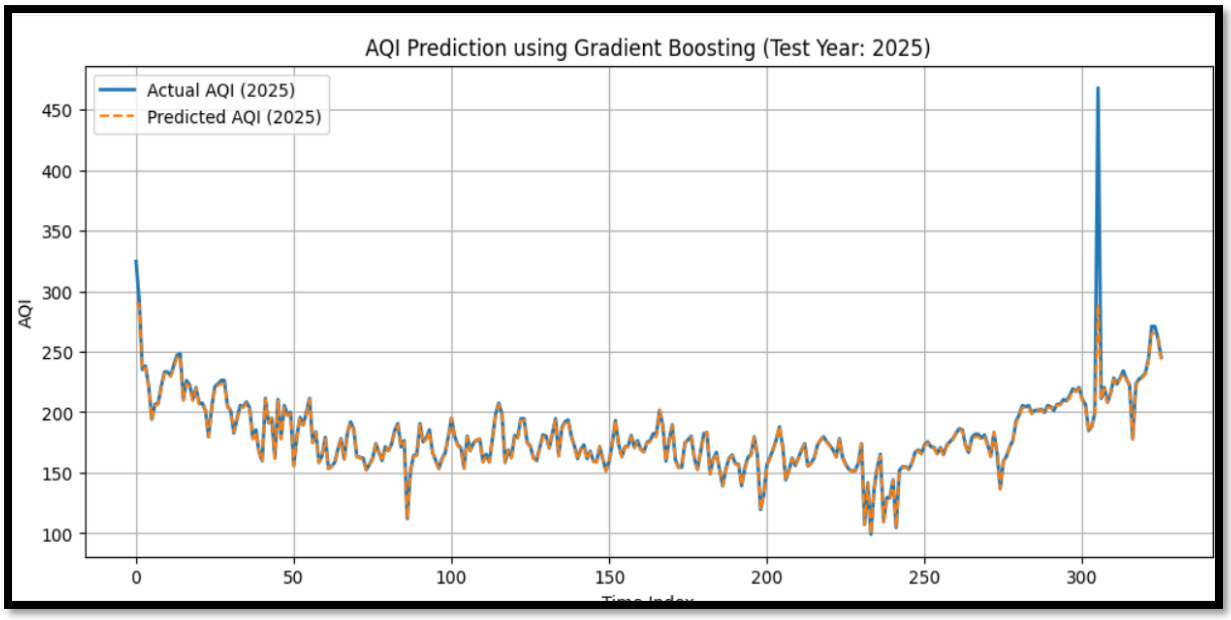


Figure 5: AQI Prediction using Gradient Boosting

Comparative Visualization:

- Comparisons of real AQI, predictions by the random forests, and Gradient Boosting predictions show:
- The models are useful in monitoring daily AQI shifts.
- Gradient Boosting is somewhat better than Random Forest in terms of its ability to follow steep AQI peaks.
- The scatter plots between each separate pollutant (PM2.5, NO 2, SO 2) and the predicted AQI confirm that the models are properly reflecting the impact of pollutants on the quality of the air.

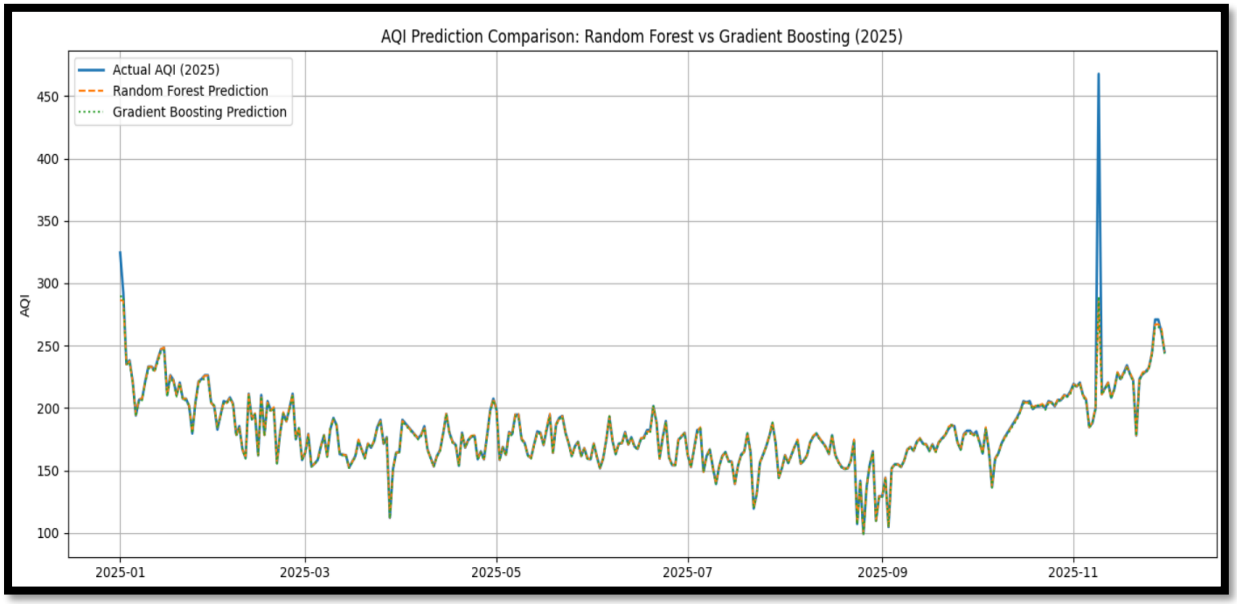


Figure 6: AQI Prediction Comparison: Random Vs Gradient Boosting

Random Forest: Actual vs Predicted

PM2.5 vs AQI

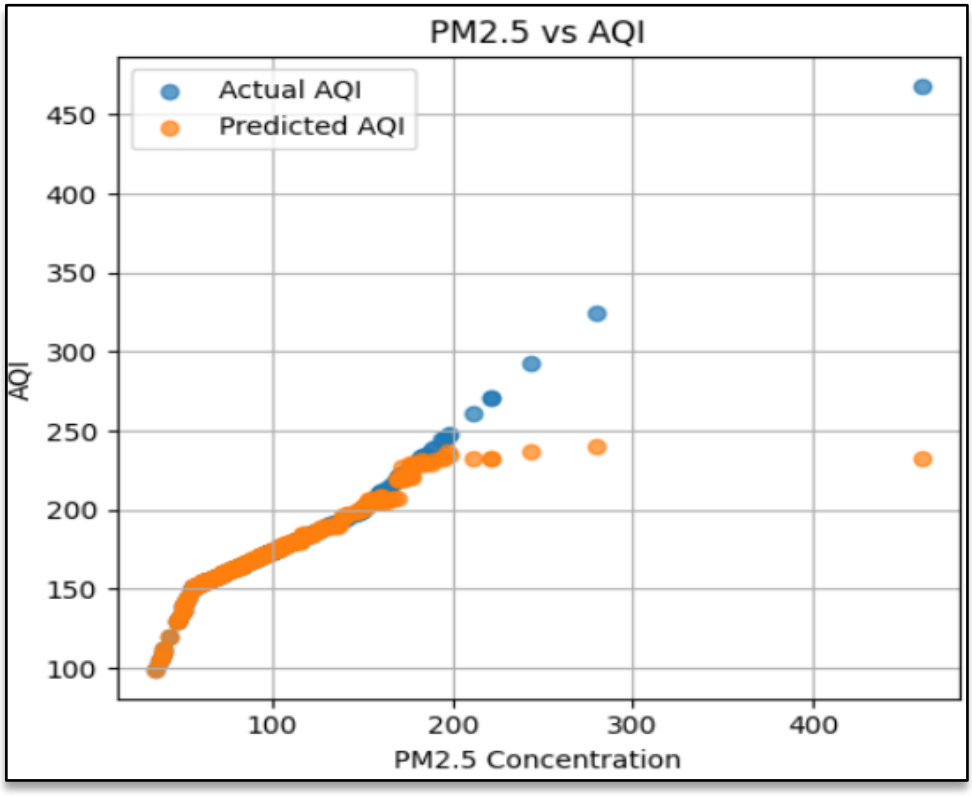


Figure 7 PM2.5 vs AQI

NO_2 vs AQI

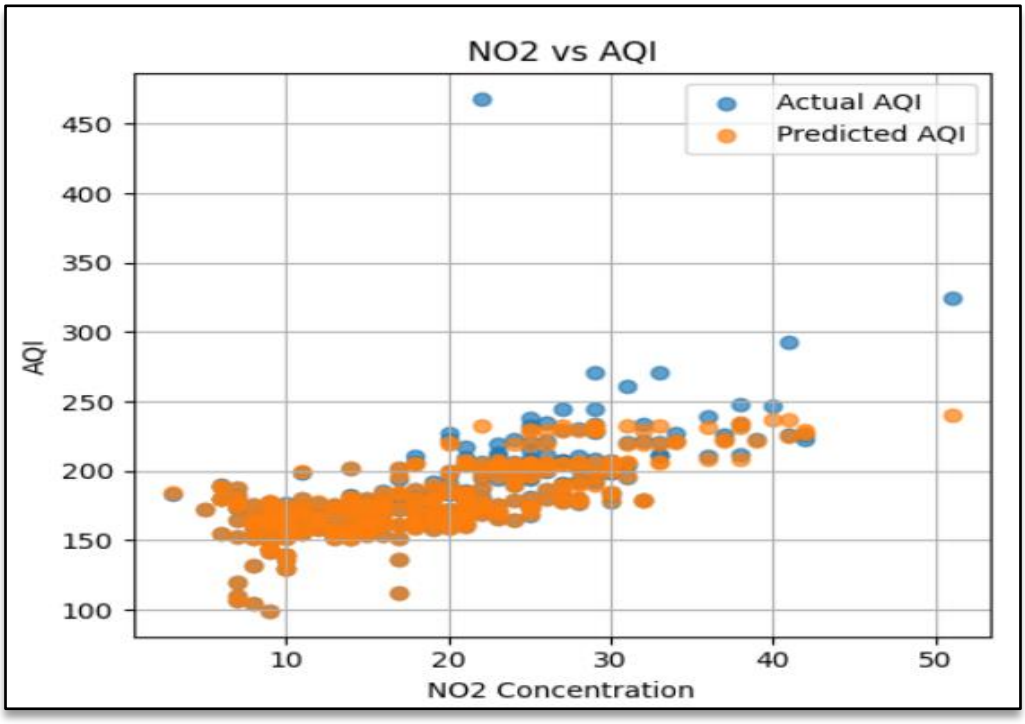


Figure 8 NO_2 vs AQI

SO_2 vs AQI

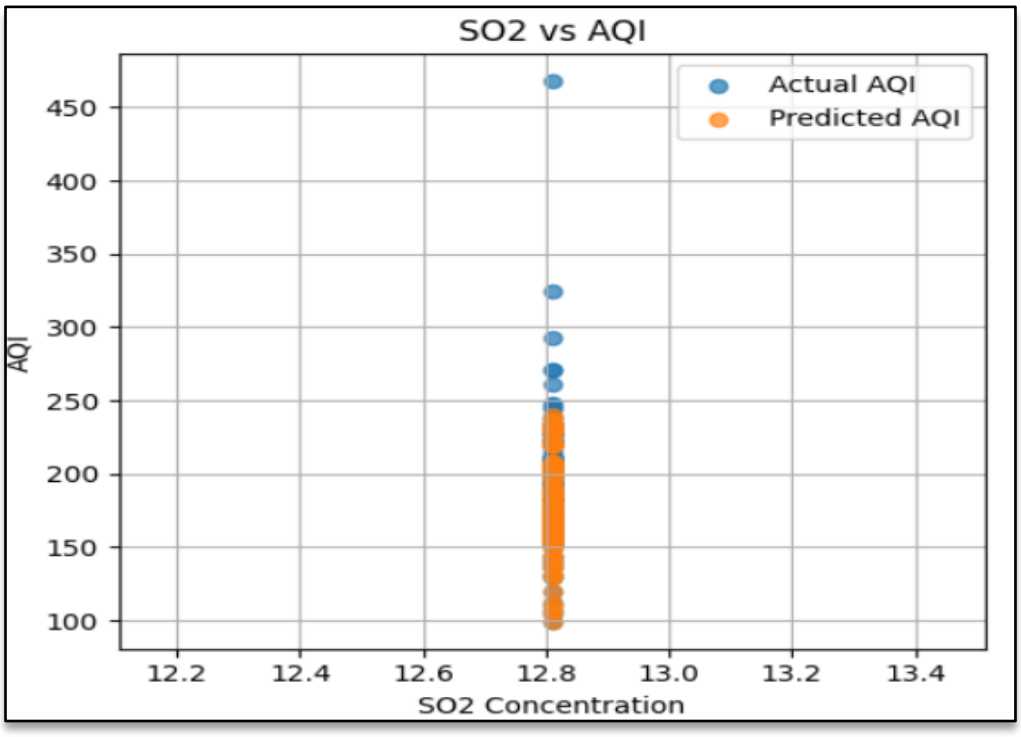


Figure 9 SO_2 vs AQI

Discussion:

The AQI prediction models were very effective in predicting the air quality. Both the Random Forest and Gradient Boosting were able to capture the day changes in pollutants though the latter was a little more accurate in the sharp peaks. The addition of lag-1 characteristics enabled the models to consider the effect of the past day pollution on the present AQI that enhanced the quality of the predictions. On the whole, the findings validate the hypothesis that past data on pollutants can be successfully applied in future air quality trend prediction.

Impact:

Proper prediction of AQI has important practical implications. The citizens are able to take early precautions when the pollution levels are high on a particular day thereby minimizing health risks. The government is able to anticipate interventions, warn and control emissions better. This predictive method assists in environmental administration, environmental health consciousness and decision guided by information, enhancing cleaner and safer urban areas.

Closing:

This project reveals a systematic method of predicting AQI with the help of Machine Learning. It offers robust and practical forecasts with the help of historical data, thorough preprocessing, feature engineering, and sophisticated modeling. The value of data-driven environmental monitoring envisioned in practice can be demonstrated through these predictions, which can, in turn, inform individuals and policymakers to enhance air quality and ensure the safety of the population.

Appendix

Appendix A: Python Code for AQI Prediction

```
# ================================
# Import Libraries
# ================================
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

from google.colab import files

# ================================
# Load Data
# ================================
df = pd.read_excel(list(uploaded.keys())[0])

# ================================
# Preprocessing
# ================================
# Convert Date to datetime
df['Date'] = pd.to_datetime(df['Date'])

# Convert pollutants to numeric
for col in ['Temperature', 'Humidity', 'NO2', 'SO2', 'PM2.5']:
    df[col] = pd.to_numeric(df[col], errors='coerce')

# Handle missing values
df.fillna(method='ffill', inplace=True)
df.fillna(method='bfill', inplace=True)

# ================================
# AQI Calculation Function
# ================================
def calculate_sub_aqi(C, breakpoints):
    for bp in breakpoints:
        if bp['C_low'] <= C <= bp['C_high']:
            return ((bp['I_high'] - bp['I_low']) /
                    (bp['C_high'] - bp['C_low'])) * (C - bp['C_low']) + bp['I_low']
    return None

# Breakpoints
pm25_breakpoints = [
    {'C_low': 0.0, 'C_high': 12.0, 'I_low': 0, 'I_high': 50},
    {'C_low': 12.1, 'C_high': 35.4, 'I_low': 51, 'I_high': 100},
    {'C_low': 35.5, 'C_high': 55.4, 'I_low': 101, 'I_high': 150},
    {'C_low': 55.5, 'C_high': 150.4, 'I_low': 151, 'I_high': 200},
    {'C_low': 150.5, 'C_high': 250.4, 'I_low': 201, 'I_high': 300},
    {'C_low': 250.5, 'C_high': 500.4, 'I_low': 301, 'I_high': 500}
]
```

```
no2_breakpoints = [
    {'C_low': 0, 'C_high': 53, 'I_low': 0, 'I_high': 50},
    {'C_low': 54, 'C_high': 100, 'I_low': 51, 'I_high': 100},
    {'C_low': 101, 'C_high': 360, 'I_low': 101, 'I_high': 150},
    {'C_low': 361, 'C_high': 649, 'I_low': 151, 'I_high': 200},
    {'C_low': 650, 'C_high': 1249, 'I_low': 201, 'I_high': 300},
    {'C_low': 1250, 'C_high': 2049, 'I_low': 301, 'I_high': 500}
]

so2_breakpoints = [
    {'C_low': 0, 'C_high': 35, 'I_low': 0, 'I_high': 50},
    {'C_low': 36, 'C_high': 75, 'I_low': 51, 'I_high': 100},
    {'C_low': 76, 'C_high': 185, 'I_low': 101, 'I_high': 150},
    {'C_low': 186, 'C_high': 304, 'I_low': 151, 'I_high': 200},
    {'C_low': 305, 'C_high': 604, 'I_low': 201, 'I_high': 300},
    {'C_low': 605, 'C_high': 1004, 'I_low': 301, 'I_high': 500}
]

# Calculate AQI columns
df['PM2.5_AQI'] = df['PM2.5'].apply(lambda x: calculate_sub_aqi(x, pm25_breakpoints))
df['NO2_AQI']   = df['NO2'].apply(lambda x: calculate_sub_aqi(x, no2_breakpoints))
df['SO2_AQI']   = df['SO2'].apply(lambda x: calculate_sub_aqi(x, so2_breakpoints))

# Final AQI
df['AQI'] = df[['PM2.5_AQI', 'NO2_AQI', 'SO2_AQI']].max(axis=1)

# Create lag features
df['PM2.5_lag1'] = df['PM2.5'].shift(1)
df['NO2_lag1']   = df['NO2'].shift(1)
df['SO2_lag1']   = df['SO2'].shift(1)

df.dropna(inplace=True)

# ================================
# Train-Test Split
# ================================
train_df = df[df['Date'].dt.year <= 2024]
test_df  = df[df['Date'].dt.year == 2025]

features = ['PM2.5', 'NO2', 'SO2', 'PM2.5_lag1', 'NO2_lag1', 'SO2_lag1']
target = 'AQI'

X_train = train_df[features]
y_train = train_df[target]

X_test = test_df[features]
y_test = test_df[target]

# ================================
# Random Forest Model
# ================================
rf_model = RandomForestRegressor(n_estimators=50, random_state=42)
rf_model.fit(X_train, y_train)
rf_pred = rf_model.predict(X_test)

# ================================
```

```
# Gradient Boosting Model
# ================================
gbr = GradientBoostingRegressor(n_estimators=200, learning_rate=0.03, max_depth=2, random_state=42)
gbr.fit(X_train, y_train)
gbr_pred = gbr.predict(X_test)

# ================================
# Evaluation Function
# ================================
def evaluate_model(y_true, y_pred, model_name):
    mae = mean_absolute_error(y_true, y_pred)
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    r2 = r2_score(y_true, y_pred)
    print(f"=== {model_name} ===")
    print(f"MAE: {mae:.2f}")
    print(f"RMSE: {rmse:.2f}")
    print(f"R² Score: {r2:.4f} ({r2*100:.2f}%)\n")

evaluate_model(y_test, rf_pred, "Random Forest")
evaluate_model(y_test, gbr_pred, "Gradient Boosting")

# ================================
# Visualization
# ================================
plt.figure(figsize=(12,6))
plt.plot(test_df['Date'], y_test, label='Actual AQI', linewidth=2)
plt.plot(test_df['Date'], rf_pred, label='Random Forest Prediction', linestyle='--')
plt.plot(test_df['Date'], gbr_pred, label='Gradient Boosting Prediction', linestyle=':')
plt.xlabel("Date")
plt.ylabel("AQI")
plt.title("AQI Prediction: Actual vs Predicted (2025)")
plt.legend()
plt.grid(True)
plt.show()

# Scatter plots for individual pollutants vs AQI
plt.figure(figsize=(15,5))

for i, pollutant in enumerate(['NO2', 'SO2', 'PM2.5']):
    plt.subplot(1, 3, i+1)
    plt.scatter(test_df[pollutant], y_test, label='Actual AQI', alpha=0.7)
    plt.scatter(test_df[pollutant], gbr_pred, label='Predicted AQI', alpha=0.7)
    plt.xlabel(pollutant)
    plt.ylabel('AQI')
    plt.title(f'{pollutant} vs AQI')
    plt.legend()
    plt.grid(True)

plt.tight_layout()
plt.show()

# ================================
# Save Predicted Data
# ================================
test_df['AQI_RF_Predicted'] = rf_pred
test_df['AQI_GBR_Predicted'] = gbr_pred
```

```
test_df.to_excel("Predicted_AQI_2025.xlsx", index=False)
files.download("Predicted_AQI_2025.xlsx")
```